

TECHNICAL UNIVERSITY

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #2

Execution trees

OF CLUJ-NAPOCA

Computer Science

Agenda

- **LP paradigms – review**
- **Operational Semantic - review**
- **Execution tree**
 - Representation
 - Mechanism

Prolog PREDICATES

(rules, facts, queries)

- **Clause** general form:

$p(X) :- q_1(Y), q_2(X, Y), \dots, q_n(X, Z) .$
= a **THEOREM** in form "conclusion if hypotheses"
meaning $q_1 \wedge q_2 \wedge \dots \wedge q_n \rightarrow p$

- **Fact**

$p(a) .$

= a clause without body

= a theorem WITHOUT any hypothesis = **AXIOM** (no need for proof)

- **Query**

$?- q_2(a, b) .$

= a clause without head

= a theorem WITHOUT conclusion

They define the **declarative semantics** of Prolog = interpretation of the statements

Execution of LP programs

Unification mechanism – the core of LP

- **Q1:** For the query/head unification take
 - A1: first clause first = top-down
- **Q2:** In the query/head successful unification the body becomes the new goal. In a conjunction of goals
 - A2: first subgoal (sub-body) first = left-right
- **Q3:** In the query/head failed unification
 - A3: unification fails = backtracking

Typical predicate (has exceptions):

```
/*general<OR>*/    p:- ... .  
                  p:- ... .  
/*specific<OR>*/   p:- q /*specific<AND>*/, ..., t /*general<AND>*/.
```

Operational semantics in action

When building the tree, we start from the initial query in the matching process. For each (sub)goal (one at a time):

- C1: current goal succeeds? If *yes*, current node built successfully & go C2, *e/se*, backtrack.
- C2: does it match a fact? If *yes*, current node is a leaf & go C3, *e/se*, push the entire body on the execution stack, and pop stack's top.
- C3: is the stack empty? If *yes*, over (the whole execution ends successfully, the execution tree becomes at this very moment the deduction tree), *e/se* pop the top of the execution stack & go C1

First conclusions

- a Prolog **program** is a set of **theorems** (complete clauses) and **axioms** (facts)
- Executing a Prolog program means **proving a new theorem** (the query/goal) from the existing ones (program)
- The execution relies on **solving** a set of **systems of linear equations**
- The number of systems = number of nodes in the deduction tree
- Each system has a number of equations equal to the number of arguments of the goal executed in the corresponding node

Main elements of Prolog

Built-in predicate call	Outcome
var(X)	if X unbound then T else F
nonvar(X)	if X bound then T else F
atom(X)	if X constant then T else F
integer(X)	if X integer then T else F
atomic(X)	if X bound and not compound then T else F
call(X)	executes X , where X is the name of a predicate used in metaprogramming, where X gets instantiated at runtime

Main elements of Prolog – contd.

- = infix operator (*equality*)
 - Prolog attempts to match (unify) the *lhs* with *rhs*
 - if successful, they are bound together from this point on
 - an un-instantiated var will become equal to ANYTHING as the unification succeeds
- == infix operator (*identity*)
 - if $X == Y$ then $X = Y$
 - if $X = Y$ then $X == Y$ is NOT MANDATORY!!!
 - an uninstantiated var will become identical to another uninstantiated variable ONLY if they are already sharing same location, otherwise FAILS
 - is unifiable (does **not** actually do the unification):
 - ?- not(not($X = Y$)).

Examples

?- X==Y .

false (fails; even if both X and Y are free variables)

?- X==X .

true, X=_some_number

?- X=Y, X==Y.

true, X=_some_number; Y =_some_number (SAME)

?- X=[a,b], Y=[a,b], X==Y. [in STANDARD Prolog]

false (although same list [a,b] is JUST same content, they are NOT shared in memory, NOT same location)

?- X=[a,b], Y=[a,b], X==Y. [in SWI Prolog]

yes (same list [a,b] → same content, constants whether atomic or compound are considered identical)

?- X=[a,b], Y=X, X==Y.

true

Examples

is infix operator; *rhs* gets evaluated and the result instantiates the *lhs* (if successful) or else fails. ONLY *lhs* may get instantiated.

X **is** some_expression

?- X **is** 2+3.
true, X=5.

?- X=2+3.
true, X=2+3.

?- 2+3 **is** X.

Arguments are not sufficiently instantiated

?- X **is** (2+2)/(4*1).
true, X=1.

?- X=2+3, Y **is** X.
true, X=2+3, Y=5.

Examples

is infix operator; *rhs* gets evaluated and the result instantiates the *lhs* (if successful) or else fails. ONLY *lhs* may get instantiated

X **is** some_expression

?- X **is** Y+1.

Arguments are not sufficiently instantiated

?- Y=2, X **is** Y+1.

true, X=3, Y=2.

?- X=Y+1, Y=2, Z **is** X.

true, X=2+1, Y=2, Z=3.

Matching rules

Goal (q)	Rule head (r)	Outcome
?- p(a).	p(b).	fails
?- p(a).	p(a).	succeeds
?- p(X).	p(a).	succeeds - X gets instantiated to a
?- p(a).	p(X).	succeeds - X gets instantiated to a
?- p(Q).	p(R).	succeeds - Q and R become the same

Execution tree representation

Tree built top-down and left-right (DFS)

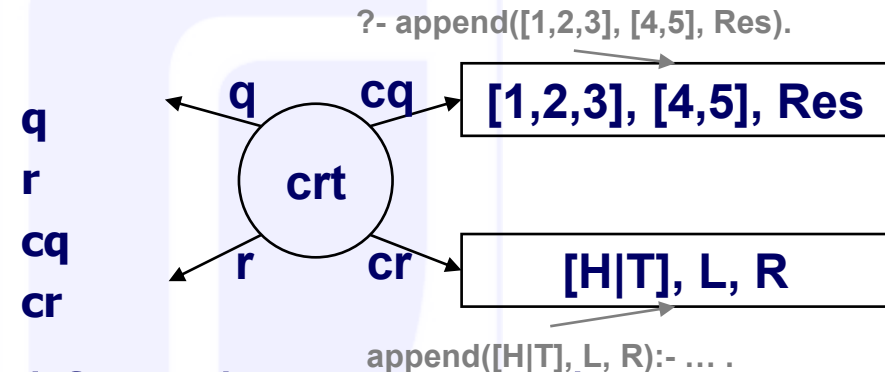
- Is a multiway tree (represented as binary)
- Each node in the exe tree contains **pointers** to several structures
 - Data: Program and memory locations of the variables (**1**)
 - Linking: connections in the tree (to navigate during execution, including backtracking) (**2**)
- Several **actions** to build the tree (**3**)

Computer Science

Execution tree: Data pointers (1)

A set of **4 pointers** to the:

- **query (goal)**
- **head of the rule**
- **context of the query (goal)**
- **context of the rule**

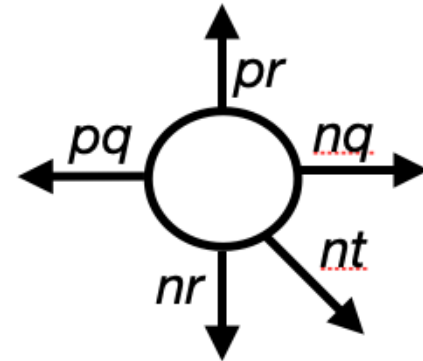


- The **query context (cq)** inherited from the parent node
 - *rule context of the parent node becomes query context of the child node* (it may be enhanced by the query context of a brother to the left due to hidden variables, or enhanced with new variables here)
 - variables of the query context are called **actual parameters**
- The **rule context (cr)** is allocated at the level of the current node
 - *ALL **formal variables** are indexed with the number of the node* (to keep track of the variable occurrence)

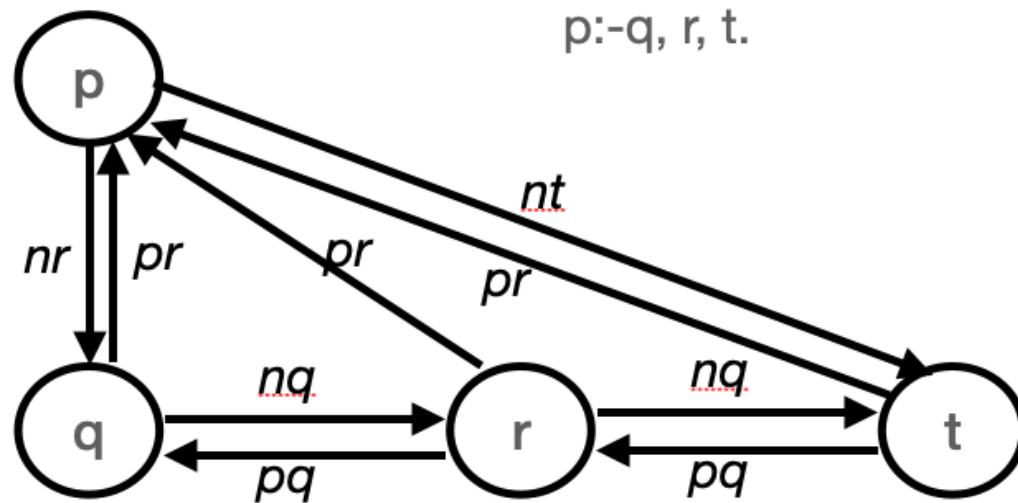
Execution tree: Linking pointers (2)

A set of **5 pointers** to other nodes in the tree (they provide the ability to make the sound actions according to the operational semantics):

- **parent rule** **pr**
 - (parent node; for root, null)
- **previous query** **pq**
 - (sibling node to the left; for first child, null)
- **next rule** **nr**
 - (first child, if any; for leaves, null)
- **next query** **nq**
 - (sibling node to the right; for last child, null)
- **next try** **nt**
 - (the next alternative to try in case of failure/backtracking; points to the rightmost child, to provide fast reachability to the last node in the tree)

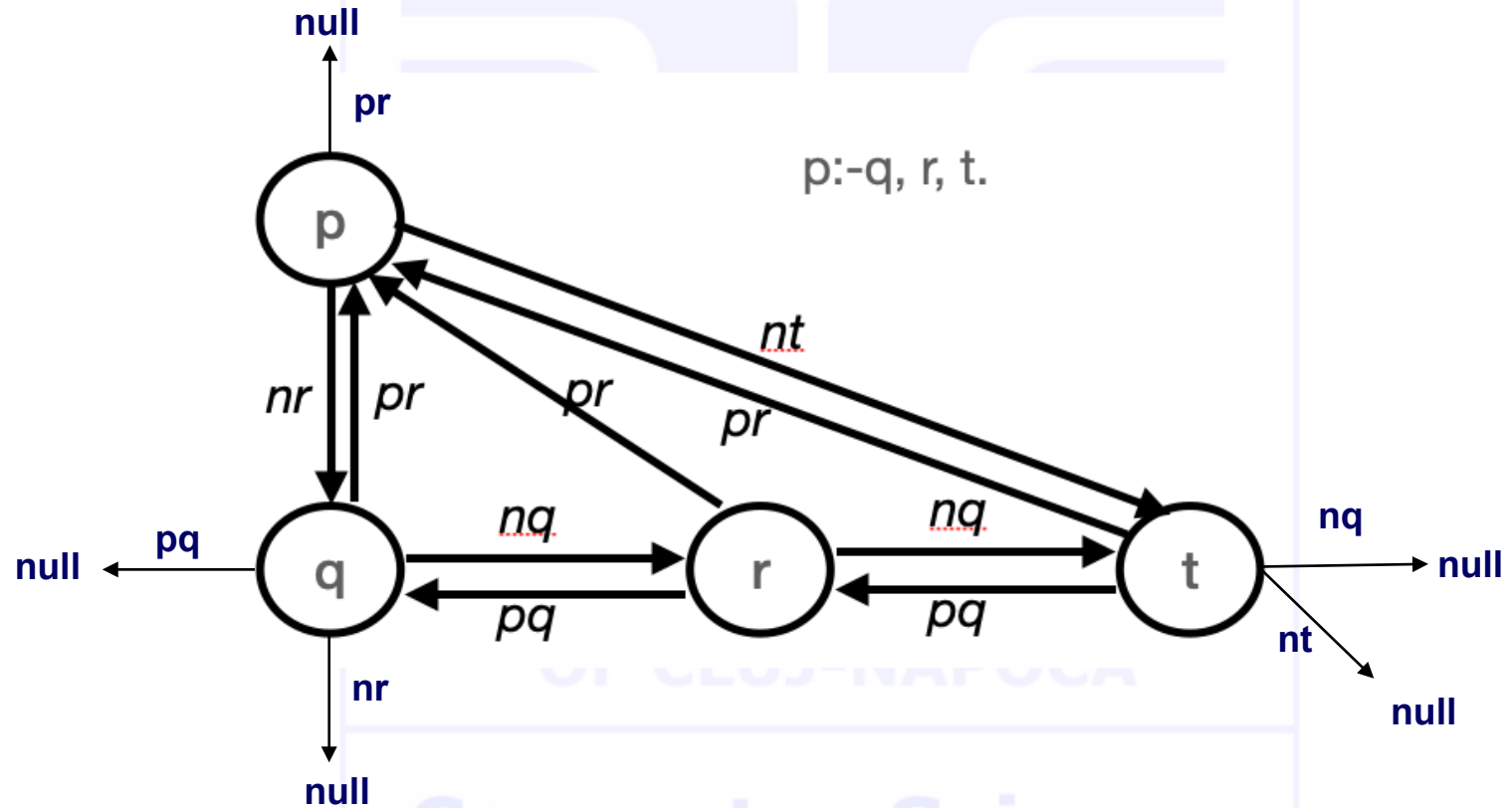


Execution tree: Linking pointers (2)

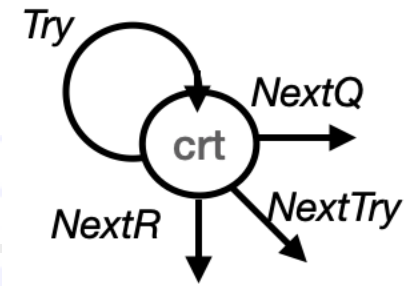


Computer Science

Execution tree: Linking pointers (2)



Execution tree: Actions (3)



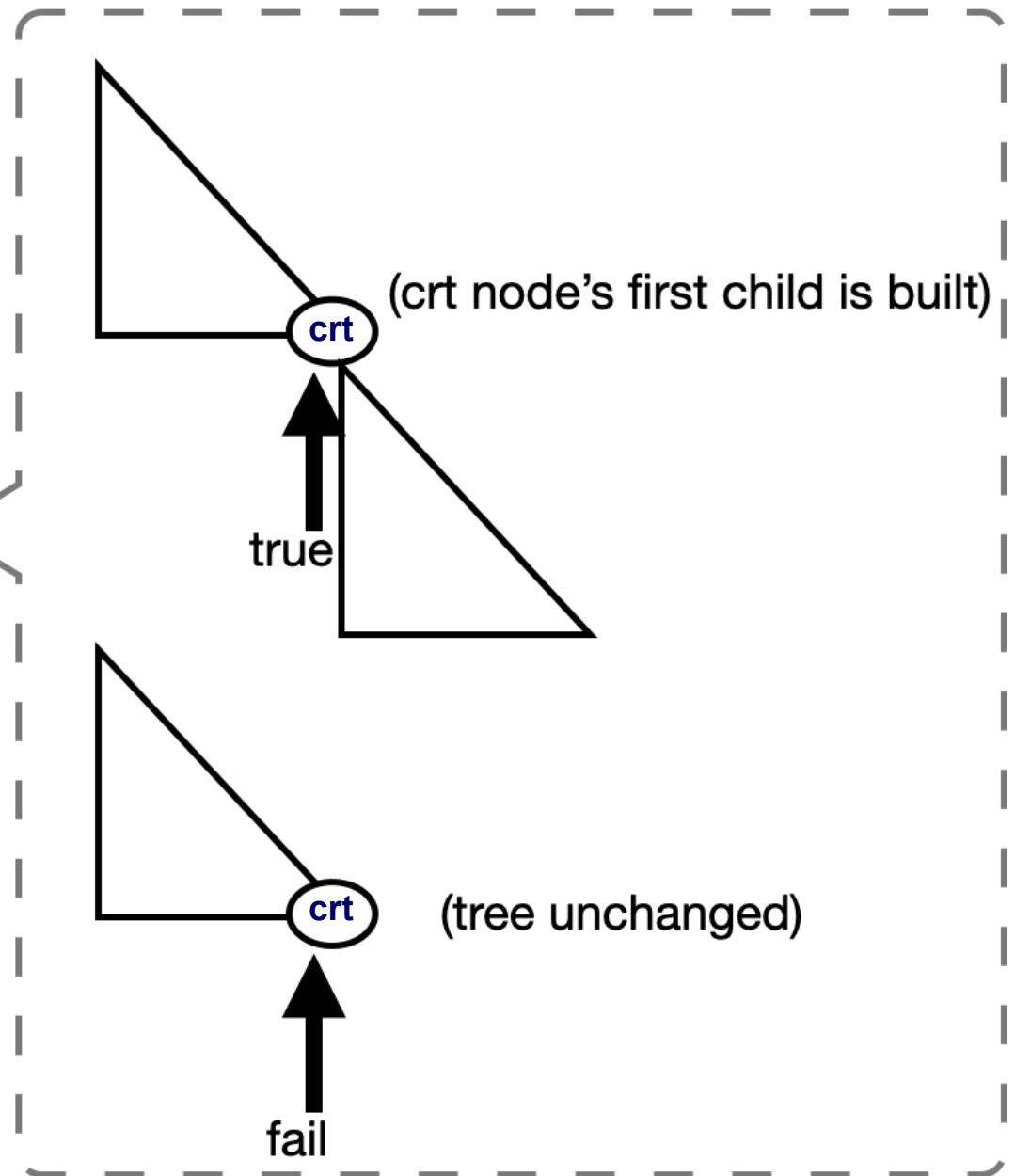
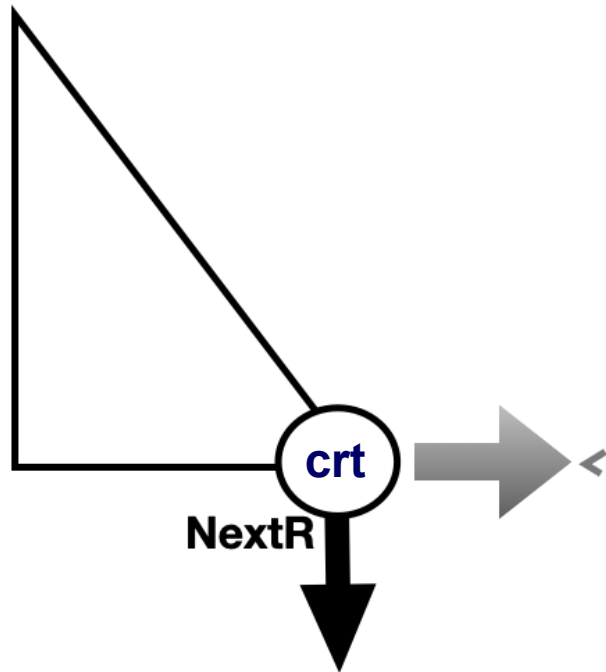
- **Try** - tries the definition of the predicate with the same name as the query (represents the matching between a query and the head of the rule)
 - the action BUILDS the **current node**
- **NextR** - enters the body of the clause at the first subgoal in the body, passing the conditional (**$:-$**)
 - the action BUILDS the **first child** of the current node
- **NextQ** - continues the body of the clause at the next subgoal in the body, passing a conjunction (**$,$**)
 - the action BUILDS the **first sibling to the right** of the current node
- **NextTry** – determines the first node to backtrack and launches the backtracking mechanism
 - the action takes place in a tree already built, by trying an alternative solution for the node identified as responsible to backtrack (examples during seminars)

Execution tree: Pointers and Actions – NextR

NextR

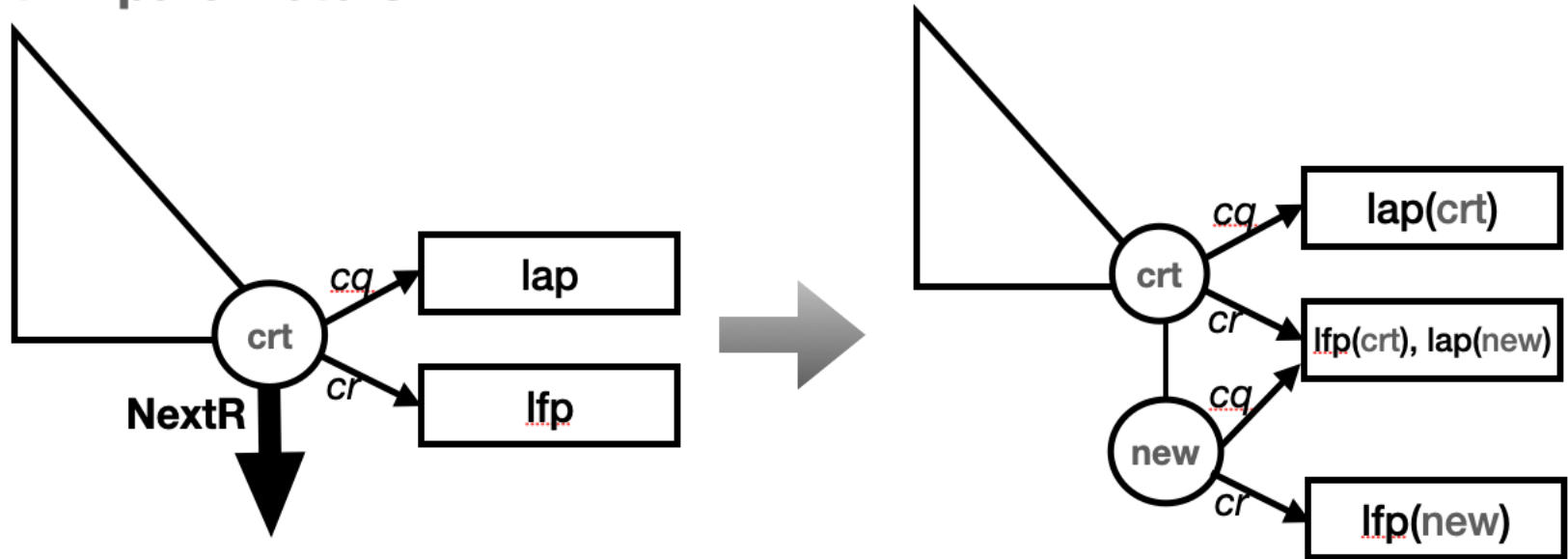
- At least one *new* node is generated, the left child of the current node
- If it (*new* node; first child of current) matches a fact, it is a leaf
- If matches a complete rule, the entire tree rooted by it (*new* node; first child of current) is generated
- the list of **actual parameters** of the new node is **inherited** from the list of formal parameters of the current node
- The list of **formal parameters** of the new node is **allocated now** (thus, the corresponding variables are indexed with the number of the node)

NextR



NextR – how arguments evolve

NextR - parameters



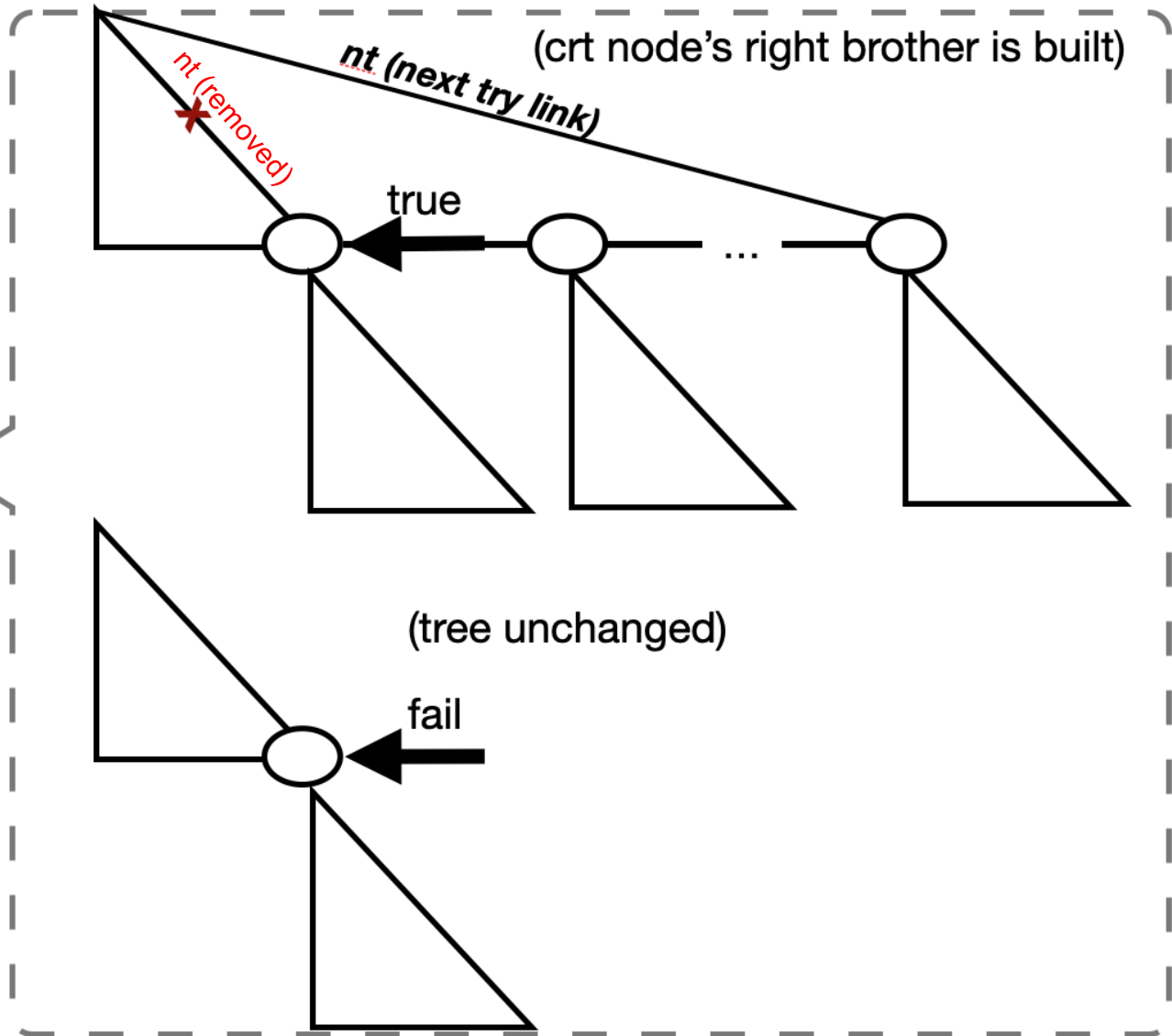
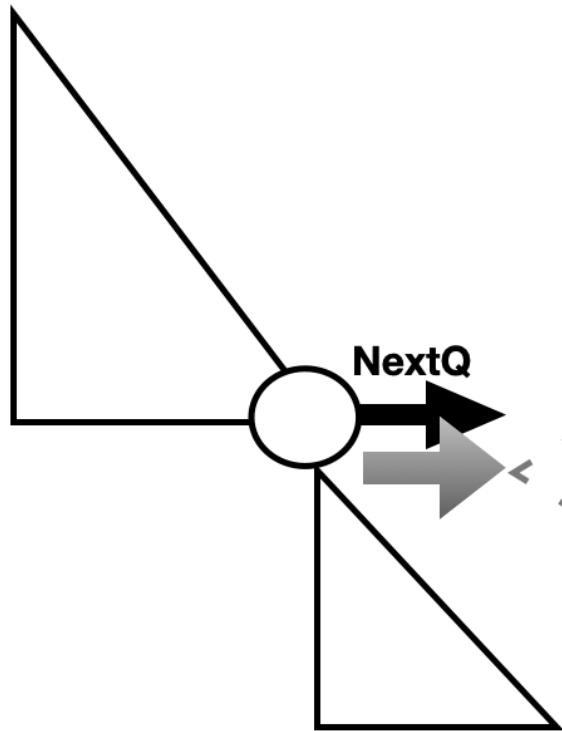
- the list of **actual parameters** of the new node is **inherited** from the list of **formal parameters** of the current node
- the list of **formal parameters** of the new node is **allocated** now (thus, the corresponding variables are indexed with the number of the node)

Execution tree: Pointers and Actions – NextQ

NextQ

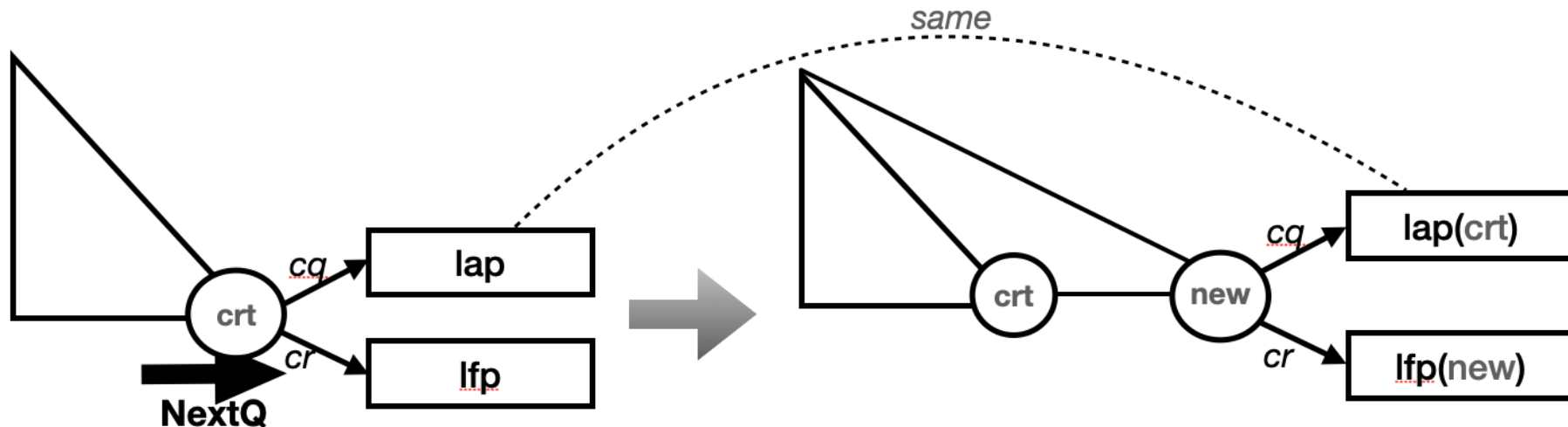
- At least one *new* node is generated, the right brother of the current node
- If it (*new* node; right brother of current)
 - matches a fact, it is a leaf;
 - else the entire tree rooted by is generated
- If it is NOT the rightmost sibling (NOT the last in conjunction, NOT before), all its right siblings (with subtrees) are generated when completed
- the list of **actual parameters** of the new node is **inherited** from the list of actual parameters of the current node
- The list of **formal parameters** of the new node is **allocated now** (thus, the corresponding variables are indexed with the number of the node)

NextQ



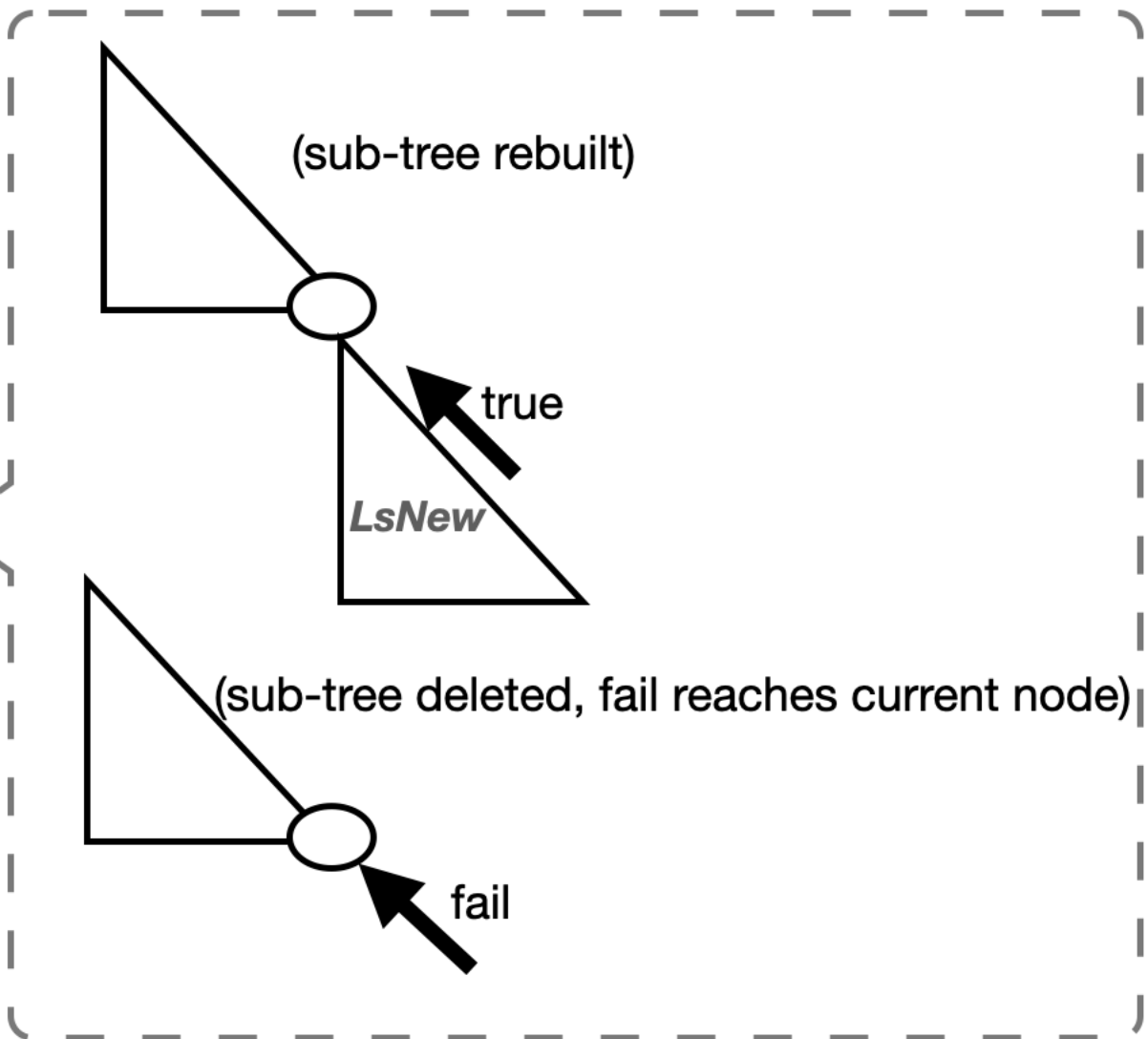
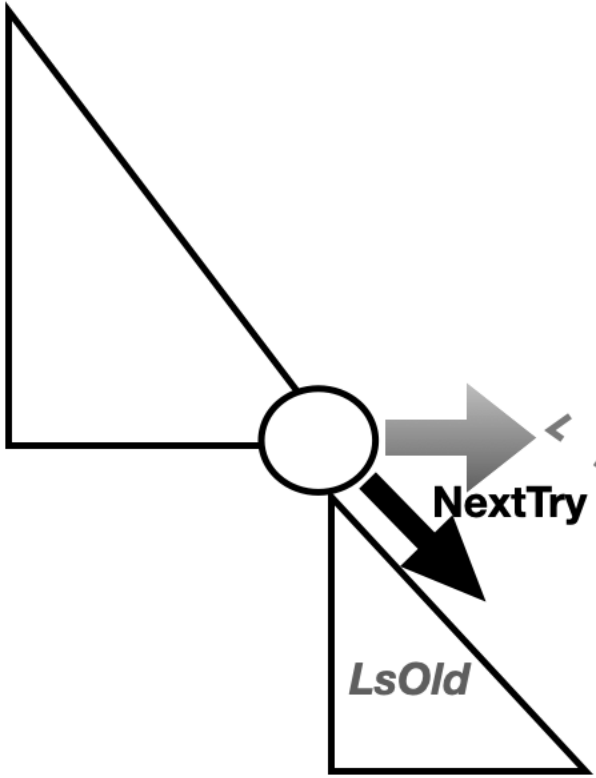
NextQ – how arguments evolve

NextQ - parameters



- the list of **actual parameters** of the new node is **inherited** from the list of **actual parameters** of the current node
- the list of **formal parameters** of the new node is **allocated** now (thus, the corresponding variables are indexed with the number of the node)

NextTry



Example 3rd

Check if an element is present in a list

How many arguments/why?

Predicate's signature is `member/2 (+Element, +List)`

```
member (X, L) :-  
    L = [H | T],  
    X = H.
```

```
member (X, L) :-  
    L = [H | T],  
    X \= H,  
    member (X, T).
```

```
member (X, L) :-  
    L = [],  
    fail.
```

Example 3rd contd.

```
member (X, L) :-
    L = [H | T],           % list not empty
    X = H.                 % searched element matches the head of the list

member (X, L) :-
    L = [H | T],           % list not empty
    X \= H,                 % searched element doesn't match the head of the list
    member (X, T).         % searched element found in the rest of the list

member (X, L) :-
    L = [],                % if reached the empty list
    fail.                  % searched element not found in the list
```

With default unifications, the predicate becomes:

```
member (X, [X | T]).       % default decomposition and unification
member (X, [H | T]) :-    % default decomposition
    X \= H,                % Is it necessary? Why/why not?
    member (X, T).

member (X, []) :-
    fail.                  % Is it necessary? Why/why not?
```

Example 3rd contd.

```
member (X, [X|T]) .      % default decomposition and unification
member (X, [H|T]) :-    % default decomposition
    X\=H,                % Is it necessary?
    member (X, T) .      % not really due to the search rule (A1)
member (X, []) :-        % Is it necessary?
    fail .               % not really due to the default failure by absence.
```

Thus, the predicate becomes:

```
member (X, [X|T]) .
member (X, [H|T])
    member (X, T) .
```

Qs:

- What happens if we reverse the order of clauses? Would it still be a correct predicate? Why/why not?
- How is better? Why?

The member/2 predicate

(1)
q/m₁

```
[m1] member(X, [X|_]).
```

```
[m2] member(X, [_|T]) :-
```

```
[m21]     member(X, T).
```

```
[q] ?- member(2, [1,2,3]).
```

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

$[m_1]$ `member(X, [X|_]) .`

$[m_2]$ `member(X, [_|T]) :-`

$[m_{21}]$ `member(X, T) .`

$[q]$ `?- member(2, [1,2,3]) .`

(1)
q/m₁

$2 = X_1$
 $[1,2,3] = [X_1|_]$

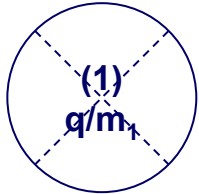
R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]     member(X, T).
```

```
[q] ?- member(2, [1,2,3]).
```



$2 = X_1$
 $[1,2,3] = [X_1|_]$ $\rightarrow$ **false**

C1 X

Computer Science

R1: clauses $\rightarrow$ top-down
 R2: subgoals $\rightarrow$ left-right
 R3: failure $\rightarrow$ backtrack

C1: q/head succeeds? $\rightarrow$ C2 or backtrack
 C2: fact? $\rightarrow$ C3 or body push stack
 C3: Stack empty? Stop or pop

The member/2 predicate

$[m_1]$ `member(X, [X|_]) .`

$[m_2]$ `member(X, [_|T]) :-`

$[m_{21}]$ `member(X, T) .`

$[q]$ `?- member(2, [1,2,3]) .`

(1')
q/m₂

$2 = X_1$
 $[1,2,3] = [_|T_1]$

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

$[m_1] \text{ member}(X, [X|_]).$

$[m_2] \text{ member}(X, [_|T]) :-$

$[m_{21}] \text{ member}(X, T).$

$[q] \text{ ?- member}(2, [1,2,3]).$

(1')
q/m₂

$2 = X_1 \rightarrow X_1 = 2$
 $[1,2,3] = [_|T_1] \rightarrow T_1 = [2,3]$



C1 ✓
C2 ✗

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

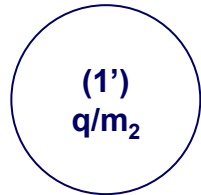
The member/2 predicate

$[m_1] \text{ member}(X, [X|_]).$

$[m_2] \text{ member}(X, [_|T]) :-$

$[m_{21}] \quad \text{member}(X, T).$

$[q] \text{ ?- member}(2, [1,2,3]).$



$2 = X_1$
 $[1,2,3] = [_|T_1] \rightarrow X_1 = 2$
 $T_1 = [2,3]$

m_{21}

$m_{21}: \text{member}(2, [2,3]).$

R1: clauses $\rightarrow$ top-down
 R2: subgoals $\rightarrow$ left-right
 R3: failure $\rightarrow$ backtrack

C1: q/head succeeds? $\rightarrow$ C2 or backtrack
 C2: fact? $\rightarrow$ C3 or body push stack
 C3: Stack empty? Stop or pop

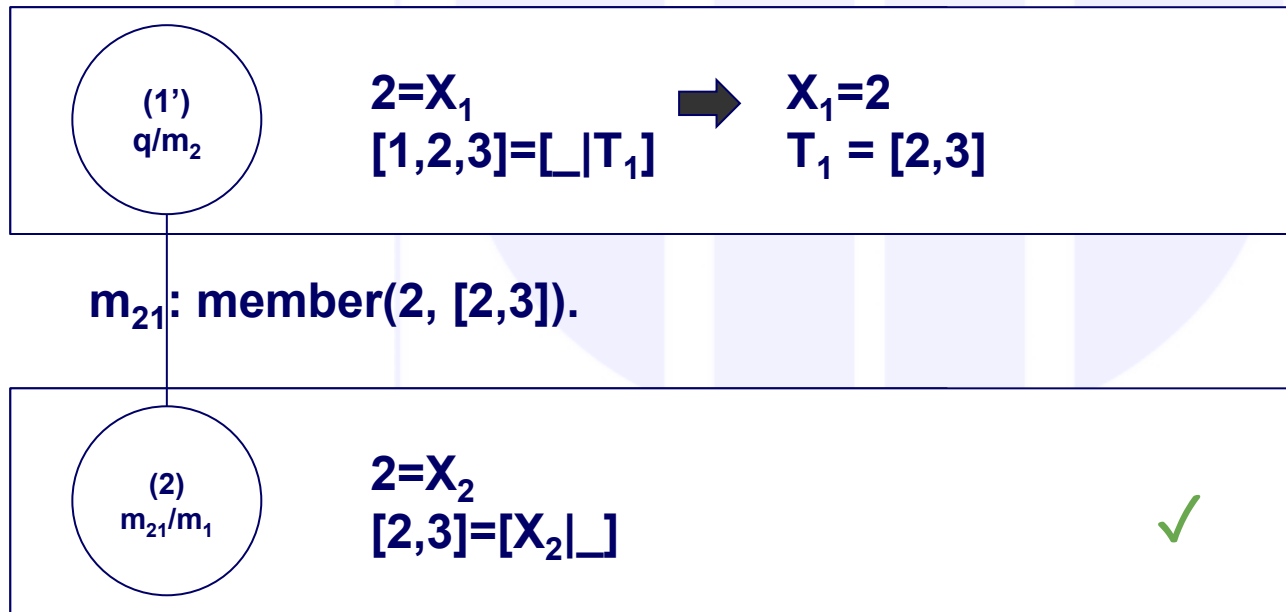
The member/2 predicate

$[m_1]$ `member(X, [X|_]) .`

$[m_2]$ `member(X, [_|T]) :-`

$[m_{21}]$ `member(X, T) .`

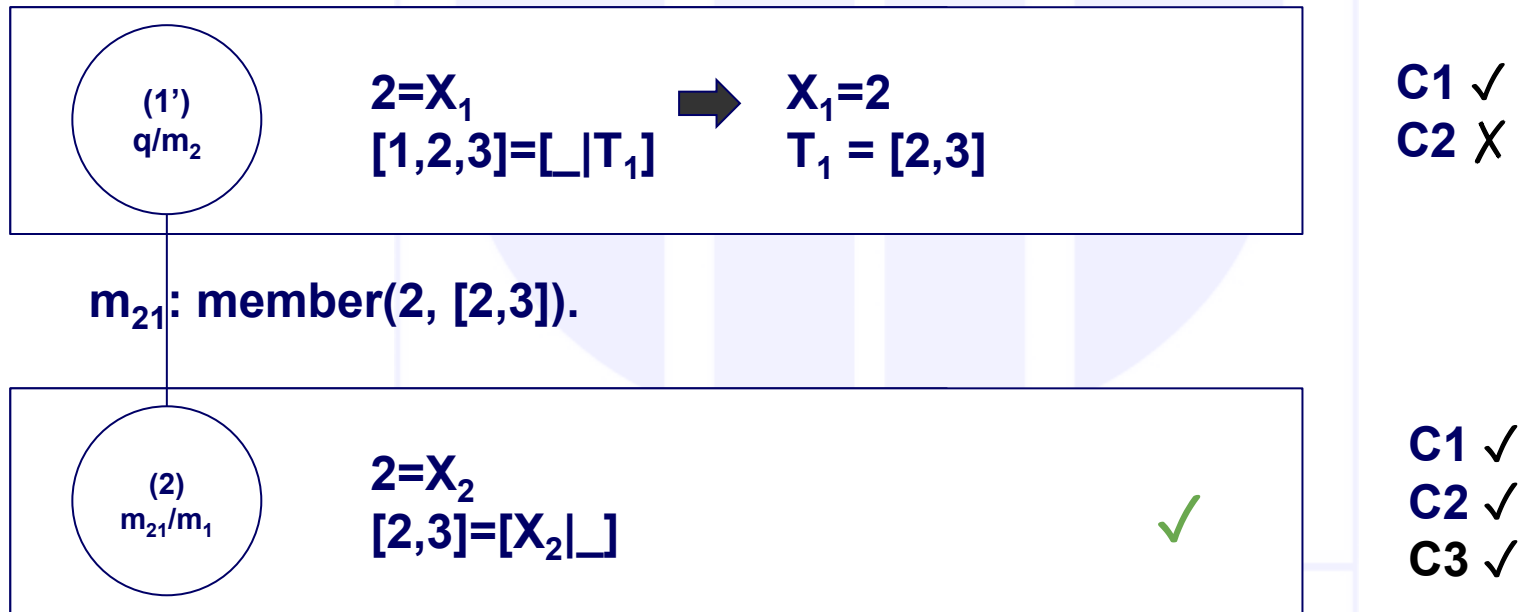
$[q]$ `?- member(2, [1,2,3]) .`



```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]     member(X, T).
```

```
[q] ?- member(2, [1,2,3]).
```

The member/2 predicate



We have overlapped with a fact, the node is a leaf,
and we do not have any other clause in the stack.

We have obtained the execution tree

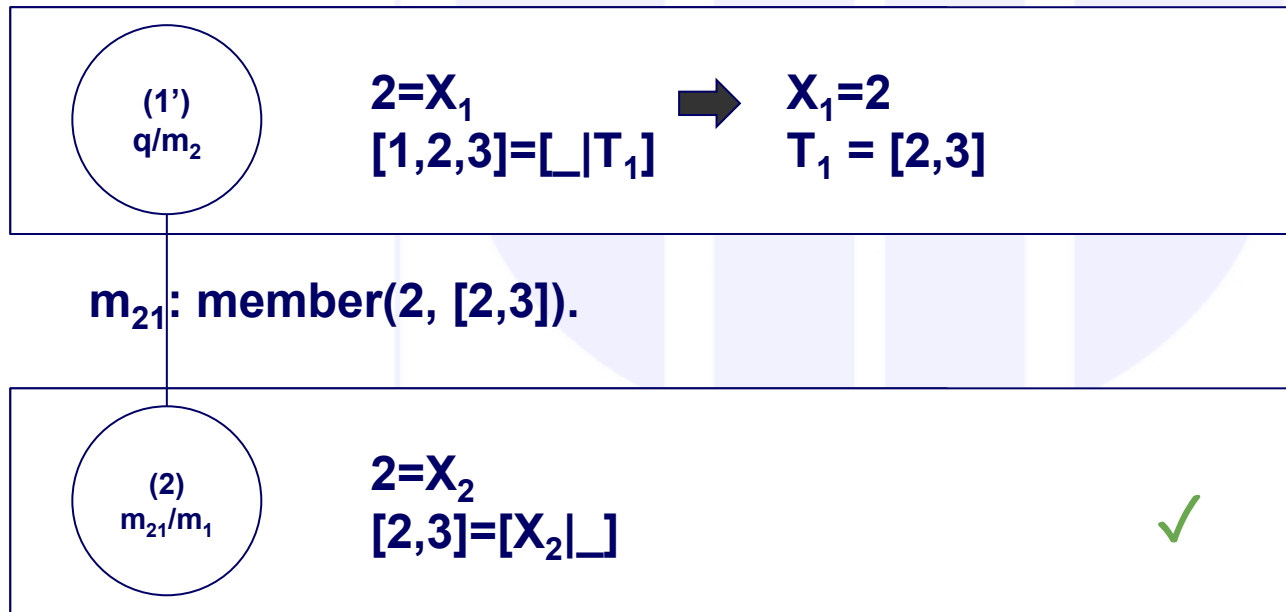
R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]      member(X, T).
```

```
[q] ?- member(2, [1,2,3]).
```

The member/2 predicate



We have overlapped with a fact, the node is a leaf
and we do not have any other clause in the stack.
We have obtained the execution tree

Do remember:

- (1) failed and only then through (1') did we get an answer.